CARGO CONVOY CRM

Developer Manual & Technical Reference

Quick Reference:
Framework: Laravel 12 (PHP 8.4)
Database: MySQL 8
Routes: 305 | Controllers: 15 | Models: 34
Migrations: 32 | Views: 123

Cargo Convoy Inc
7119 Pennsylvania Ave, Upper Darby, PA 19082
+1 (267) 513-0604 | MC: 01512751

Version 1.0 | July 2026
For Internal Development Team Only

Table of Contents

1. Project Setup & Installation
2. Architecture & Directory Structure
3. Request Lifecycle
4. Controllers Reference
5. Models & Relationships
6. Middleware & Auth
7. Services & Helpers
8. Database Schema
9. Routes & API
10. Views & Frontend
11. Credit System (Technical)
12. Email & PDF System
13. Deployment & CI/CD
14. Testing
15. Troubleshooting
16. Code Conventions

1. Project Setup & Installation

1.1 Requirements

Software	Version	Purpose
PHP	^8.2 (8.4 recommended)	Application runtime
MySQL	8.0+	Database
Composer	2.x	PHP dependency manager
Node.js	18+ (optional)	Frontend build tools
Git	2.x	Version control

1.2 PHP Extensions Required

```
php -m | grep -E "(pdo_mysql|mbstring|openssl|tokenizer|xml|ctype|json|bcmath|gd|zip|curl|dom)"
```

1.3 Local Setup Steps

```
# 1. Clone repository
git clone https://github.com/amrentech1230/crm.git
cd crm

# 2. Install PHP dependencies
composer install

# 3. Setup environment
cp .env.example .env
php artisan key:generate

# 4. Configure .env
# DB_DATABASE=ccicrm
# DB_USERNAME=root
# DB_PASSWORD=your_password
# APP_URL=http://localhost:8000

# 5. Run migrations
php artisan migrate

# 6. Create storage symlink
php artisan storage:link

# 7. Start development server
php artisan serve
```

1.4 Key Dependencies (composer.json)

Package	Version	Purpose

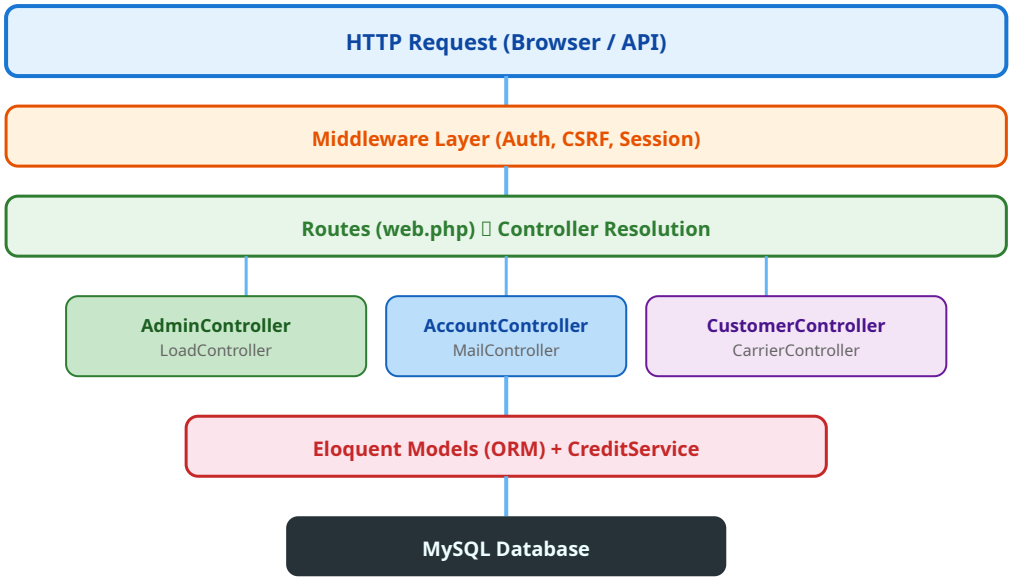
laravel/framework	^12.0	Core framework
dompdf/dompdf	*	PDF generation (BOL, invoices)
maatwebsite/excel	^3.1	Excel export for reports
guzzlehttp/guzzle	*	HTTP client
google/apiclient	^2.0	Google API integration
symfony/process	^7.3	Shell command execution

1.5 Environment Variables

```
# Application
APP_ENV=local|staging|production
APP_DEBUG=true|false
APP_URL=https://your-domain.com

# Database
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=ccicrm

# Mail (Production)
MAIL_MAILER=smtp
MAIL_HOST=smtp.your-provider.com
MAIL_PORT=587
MAIL_USERNAME=ar@cargoconvoy.co
MAIL_PASSWORD=xxxxx
MAIL_FROM_ADDRESS=ar@cargoconvoy.co

# Session
SESSION_DRIVER=database
SESSION_LIFETIME=120
```

2. Architecture & Directory Structure

2.1 Application Architecture

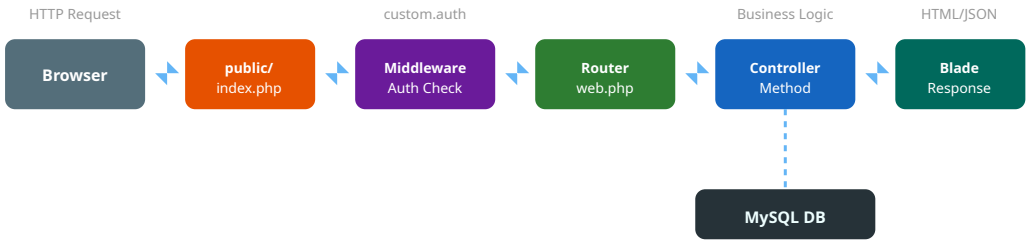


2.2 Directory Structure

crm/
├── **app/**
│ ├── **Helpers/** — *helpers.php (global functions)*
│ ├── **Http/**
│ │ ├── **Controllers/** — *15 controllers*
│ │ └── **Middleware/** — *EnsureUserIsAuthenticated, RedirectIfAuthenticated*
│ ├── **Models/** — *34 Eloquent models*
│ ├── **Providers/** — *AppServiceProvider.php*
│ └── **Services/** — *CreditService.php*
├── **config/** — *app.php, database.php, mail.php, etc.*
├── **database/**
│ ├── **migrations/** — *32 migration files*
│ ├── **factories/**
│ └── **seeders/**
├── **public/** — *Web root (assets, uploads, index.php)*
│ ├── **assets/** — *CSS, JS, libs, fonts, images*
│ ├── **uploads/** — *User-uploaded files*
│ └── index.php — *Entry point*
├── **resources/views/** — *123 Blade templates*
│ ├── **accounts/** — *Accounting views + partials*
│ ├── **admin/** — *Admin panel views*
│ ├── **broker/** — *Broker portal views*
│ ├── **auth/** — *Login/error pages*
│ └── **layout/** — *Master layout + partials*
├── **routes/**
│ └── web.php — *305 routes (all web, no API)*
├── **.github/workflows/** — *CI/CD pipelines*
├── composer.json
└── .env.example

3. Request Lifecycle

3.1 How a Request Flows



3.2 Middleware: EnsureUserIsAuthenticated

```
// Checks URL segment against user's role:
$admin    = [1, 2, 3, 22];      // Can access: admin/, broker/, account/
$accounts = [4-18, 24];          // Can access: account/, broker/
$broker   = [19, 20, 21];       // Can access: broker/ only

// If URL segment not in allowed list  abort(404)
```

3.3 View Composer (Global Menu)

```
// AppServiceProvider.php  boot()
View::composer('*', function ($view) {
    $roleId = Auth::user()->role_id;
    $menusdata = RoleHasPermission::where('role_id', $roleId)
                            ->where('read', 1)->get();
    $view->with('userMenus', $menus);  // Available in ALL views
});
```

4. Controllers Reference

4.1 Controller List

Controller	Lines	Responsibility
LoadController	~1800	Load CRUD, clone, BOL, credit check, carrier suggest
AccountController	~6500	AR accounting, invoicing, compliance, reporting, vendor, credit
AdminController	~2800	Users, roles, master data, admin load edit, IT tickets
CustomerController	~600	Customer CRUD, remittance, approval form

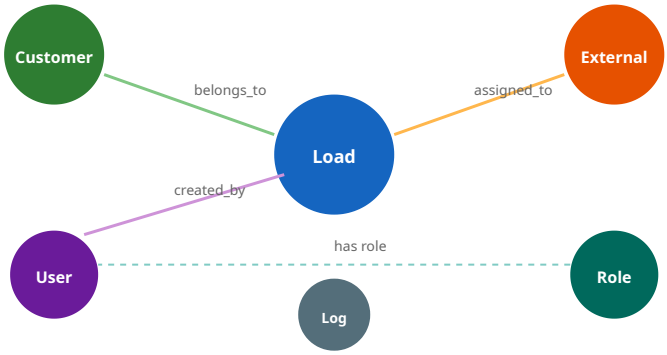
CarrierController	~400	Carrier CRUD (externals table)
ShipperController	~300	Shipper CRUD
ConsigneeController	~300	Consignee CRUD
MailController	~220	Email sending with PDF merge
FilesUploadController	~200	File upload, merge PDFs
LoginController	~150	Auth: login, logout, session
RolesController	~200	Roles & permissions management
BackupController	~50	Database backup download

4.2 Key Controller Methods

```
// LoadController — Most Important Methods:
create(Request)          → Load creation + credit validation
BrokerLoadUpdate($id)    → Load edit + credit delta + DB transaction
cloneLoad($id)           → Duplicate a load
editload($id)            → Show edit form
checkRemaingLimit()      → AJAX credit check endpoint
generateBolPdf($id)      → Download BOL as PDF
fetchCarrierSuggestions() → AJAX carrier auto-complete
```

5. Models & Relationships

5.1 Core Models



5.2 All 34 Models

Core Business

- Load
- Customer
- External (Carrier)
- Shipper

Users & Auth

- User
- Role
- Permission
- RoleHasPermission

Organization

- Department
- Office
- Manger
- TeamLeader

- Consignee
- InvoiceEmail

- PasswordResets
- IpConfig

- ProfileData

Master Data

- Country
- State
- City
- Subregion
- EquipmentType
- ShipmentType
- StatusType

Operations

- CarrierVerification
- McCheck
- Factoring
- Cmt
- ItHardware

System

- Log
- LogActivity
- Notification
- Chat
- CustomerApprovalForm

6. Middleware & Authentication

6.1 Custom Middleware

Middleware	Applied To	Logic
custom.auth	All authenticated routes	Checks session + validates URL segment against role permissions
guest	Login page only	Redirects authenticated users away from login

6.2 Authentication Flow

```
// LoginController@loginuser
1. Validate email + password
2. Check user status == 'active'
3. Check no duplicate session (sessions table)
4. Auth::attempt($credentials)
5. Log activity (IP, timestamp)
6. Redirect based on role_id  appropriate dashboard
```

7. Services & Helpers

7.1 CreditService (app/Services/CreditService.php)

Centralized credit logic used by all controllers. Registered as singleton in AppServiceProvider.

```
// Key methods:
getAvailableCreditLimit($customer)      float: current available credit
validateCreditForLoad($customer, $amt)  ['allowed' => bool, 'message' => ...]
reserveCreditForLoad($customer, $amt)   Atomic: validate + deduct with lockForUpdate
```

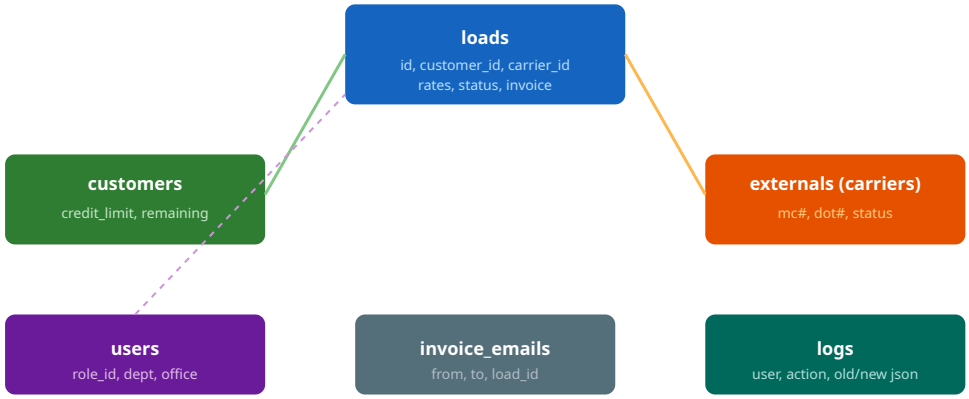
```
applyEditCreditDelta($customer, $delta)  → Atomic: apply +/- delta on load edit
calculateCustomerCreditSummary(...)      → ['assigned', 'used', 'remaining']
```

7.2 Helpers (app/Helpers/helpers.php)

```
// Auto-loaded via composer.json "files" array
addToLog($customerId, $loadId, $subject, $oldData, $newData)
  → Logs every action to logs table

calculate_customer_credit_summary($customer, $limit, $loadAmt, $received)
  → Returns credit breakdown array

getDifference($old, $new)
  → Used in load edit to compute rate delta
```

8. Database Schema

8.1 Core Tables ER Diagram



8.2 Loads Table — Key Columns

Column	Type	Purpose
id / load_number	bigint	PK, also used as load_number
customer_id	bigint	FK → customers.id
user_id	bigint	FK → users.id (creator)
load_status	varchar	Open/Delivered/Completed/Cancelled
invoice_status	varchar	null/Paid/Paid Record
shipper_load_final_rate	float	Customer's total rate
load_final_carrier_fee	float	Carrier's total fee
load_shipperr	JSON	Array of shipper objects

load_consignee	JSON	Array of consignee objects
shipper_load_other_charge	JSON	Extra charges [{type, amount, for_invoice}]
public_file	JSON	Array of uploaded doc paths
bol_edit_data	JSON	Saved BOL edits for PDF generation

8.3 Customers Table — Credit Columns

Column	Type	Note
adv_customer_credit_limit	float	Total assigned limit (from logs sum)
remaining_credit	float	Balance left (server-computed only)
remaining_credit_amount	varchar(100)	Synced copy of remaining_credit
credit_limit_log	longtext	JSON array of {credit_limit, credit_time}
remaining_credit_logs	longtext	JSON audit of balance changes
invoice_credit_limit	longtext	Separate invoice limit

9. Routes & API

9.1 Route Organization

All 305 routes are in `routes/web.php` inside a single `middleware('custom.auth')` group. No separate API routes file.

```
Route::middleware('custom.auth')->group(function () {
    // Admin routes:  /admin/*    (~80 routes)
    // Account routes: /account/* (~120 routes)
    // Broker routes:  /broker/*  (~60 routes)
    // Shared routes:  /files/*, /remaing/* (~10 routes)
});
```

9.2 Route Naming Convention

Pattern	Example	Controller
admin/{action}	admin/create-user	AdminController
account/{module}	account/accounting	AccountController
broker/{entity}	broker/load	LoadController
broker/{entity}-create	broker/load-create	LoadController@create

10. Views & Frontend

10.1 Layout Structure

```
resources/views/layout/compact/app.blade.php   Master layout
    @include('layout.partials.header')         Topbar + Sidebar
    @yield('content')                          Page content
    @include('layout.partials.footer')         Footer + JS libs

// All pages extend: @extends('layout.compact.app')
```

10.2 Frontend Stack

Library	Version	Usage
Bootstrap	5.x	CSS framework, grid, components
jQuery	3.6	DOM manipulation, AJAX
Select2	4.1	Searchable dropdowns
DataTables	1.x	Server-side tables with sort/search
ApexCharts	3.x	Dashboard charts (reporting)
TinyMCE	6.8	Rich text editor

> **No build step:** All assets are pre-compiled in `public/assets/`. No Vite/Webpack. CSS/JS loaded directly via `asset()` helper.

11. Credit System (Technical Deep Dive)

11.1 Credit Deduction on Load Create

```
// LoadController@create() — Simplified flow:
1. Validate request (rate > 0, customer exists)
2. Build Load model with all fields
3. Call CreditService::reserveCreditForLoad($customer, $amount)
     DB::transaction with lockForUpdate
     Check: available >= amount
     Deduct: remaining_credit -= amount
     Sync: remaining_credit_amount = remaining_credit
4. If credit check fails  return error, load NOT saved
5. If passes  save load, log activity
```

11.2 Credit Restoration on Load Cancel

```
// AdminController::applyCancelledLoadAccounting()
$finalRate = $originalLoad->shipper_load_final_rate;
```

```
$invoiceCredit = min(invoiceCreditAmount($load), $finalRate);
$actualCredit = max($finalRate - $invoiceCredit, 0);

$customer->remaining_credit += $actualCredit;
$customer->invoice_credit_limit += $invoiceCredit;
```

11.3 Credit Delta on Load Edit

```
// BrokerLoadUpdate — wrapped in DB::transaction
$oldRate = $request->old_shipper_load_final_rate;
$newRate = $request->shipper_load_final_rate;
$delta = $oldRate - $newRate;  // positive = credit freed

if (remaining_credit + delta < 0)  BLOCK
else  remaining_credit += delta
```

12. Email & PDF System

12.1 MailController Flow

```
// MailController@send()
1. Validate: email required, documents[] optional
2. resolveEmailSubject()  custom or default
3. prepareAttachments($files, $loadNo):
   a. Validate each file path is under allowed directories
   b. If single file  attach directly
   c. If multiple  merge all into ONE PDF via FPDI
   d. Images  converted to PDF via Dompdf first
4. Mail::mailer('smtp')->raw('', callback)
5. Cleanup temporary merged files in finally{}
```

12.2 PDF Generation (BOL)

```
// LoadController@generateBolPdf($id)
$load = Load::find($id);
$editData = json_decode($load->bol_edit_data, true);
$html = view('broker.bol_pdf', compact('load', 'editData'))->render();
$dompdf = new Dompdf();
$dompdf->loadHtml($html);
$dompdf->setPaper('letter', 'portrait');
$dompdf->render();
return $dompdf->stream("BOL_{$load->load_number}.pdf");
```

13. Deployment & CI/CD

13.1 Deployment Architecture

Deploy Production

13.2 Deploy Script (What happens on each deploy)

```
cd $DEPLOY_PATH
git fetch --all --prune
git reset --hard origin/$BRANCH
chmod -R a+rX public            # □ CRITICAL: fixes CSS 404
composer install --no-dev
php artisan down --retry=15
php artisan migrate --force
php artisan optimize:clear
php artisan config:cache
php artisan view:cache
php artisan storage:link
php artisan queue:restart
php artisan up
```

13.3 Environments

Environment	Branch	URL	Deploy
Staging	devN	https://ccicrm.in	Auto on push
Production	main	https://crmcargoconvoy.co	Manual approval

> **CSS Not Found Fix:** Always run `chmod -R a+rX public` after deploying. The server's umask makes files unreadable by nginx otherwise.

14. Testing

14.1 Test Setup

```
php artisan test            # Run all tests
php artisan test --filter=Credit  # Run credit tests only
```

14.2 Key Test Areas

Area	What to Test
Credit validation	Load creation blocked when credit exceeded
Load lifecycle	Status transitions (Open□Delivered□Completed□Paid)
Rate calculations	Final rate = base + FSC + charges
Auth middleware	Role-based URL access blocked/allowed
PDF generation	BOL PDF renders without errors

15. Troubleshooting

Issue	Cause	Fix
CSS/JS 404 on server	File permissions after deploy	`chmod -R a+rX public`
Credit goes negative	Missing server-side check	Ensure CreditService is used in all controllers
Session conflict	Multiple logins same user	Clear sessions table or wait 1 min timeout
Migration fails	Table already exists from SQL dump	Insert migration name into migrations table
route:cache fails	Duplicate route names in web.php	Don't run route:cache until duplicates fixed
Composer OOM	Server low on RAM	`COMPOSER_MEMORY_LIMIT=-1 composer install`
Mail not sending	SMTP config wrong	Check .env MAIL_* settings, test with artisan tinker
PDF merge fails	FPDI packages missing	`composer require setasign/fpdi setasign/fpdi-fpdf`

16. Code Conventions

16.1 General Rules

- Follow PSR-12 coding standard (use Laravel Pint)
- All credit operations must go through `CreditService`
- Never trust client-side input for financial fields
- Always wrap credit mutations in `DB::transaction` + `lockForUpdate`
- Log every significant action via `addToLog()`
- Use `$request->validate()` for all form inputs

16.2 Git Workflow

```
Branches:
  main      → Production (protected, requires PR + approval)
  devN      → Staging (auto-deploys)
  devSumit  → Developer branch (Sumit)
  devNeha   → Developer branch (Neha)
  feature/* → Feature branches

Commit format:
  feat: Add aging close feature
```

```
fix: Prevent credit bypass on load create
docs: Update user manual
```

Cargo Convoy CRM - Developer Manual v1.0

Confidential - Internal Development Team Only | July 2026